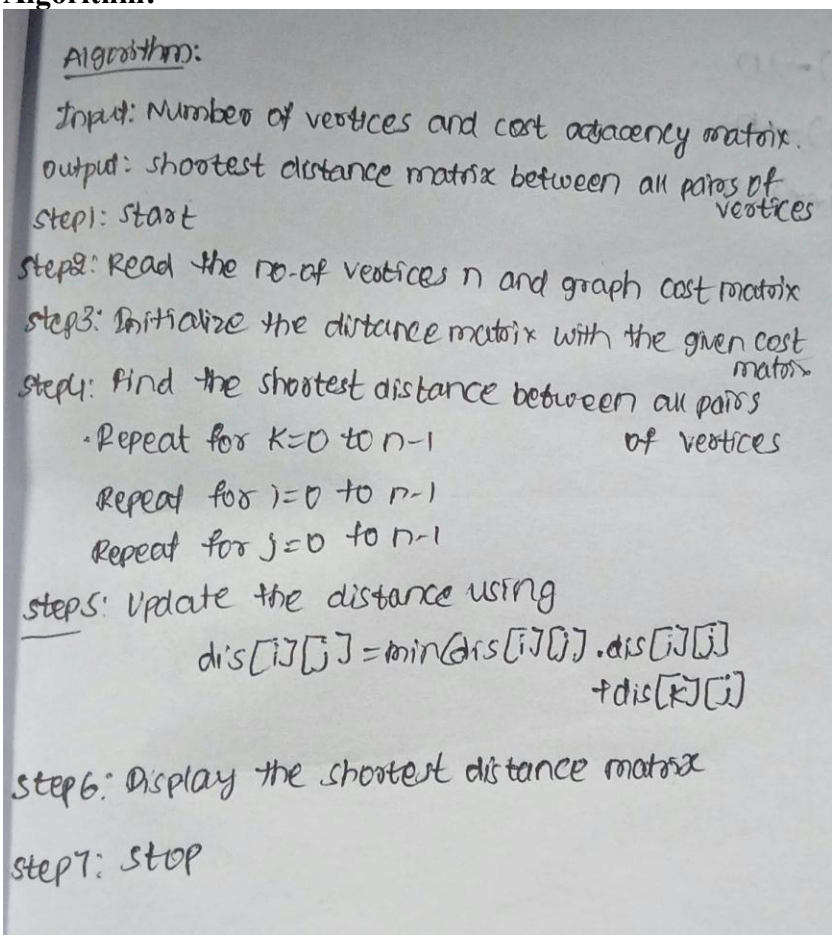
Practical-11

Aim: To Implement of Floyd-Warshal algorithm.

Floyd-Warshal algorithm

Algorithm:



Program:

```
#include <iostream>
using namespace std;

#define INF 99999

int main()
{
   int n, i, j, k;
   int dist[10][10];

   cout << "Enter number of vertices: ";
   cin >> n;
```

```
    cout << "Enter adjacency matrix (use 99999 for INF):\n";

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            cin >> dist[i][j];
        }
    }

    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    cout << "Shortest distance matrix:\n";

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (dist[i][j] == INF)
                cout << "INF ";
            else
                cout << dist[i][j] << " ";
        }
        cout << endl;
    }

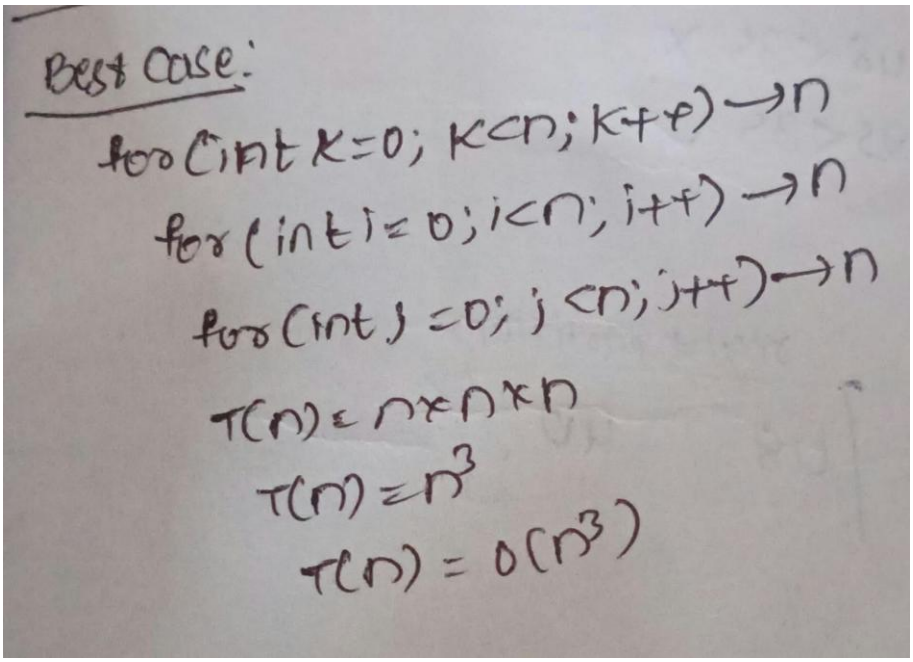
    return 0;
}
```

Output:

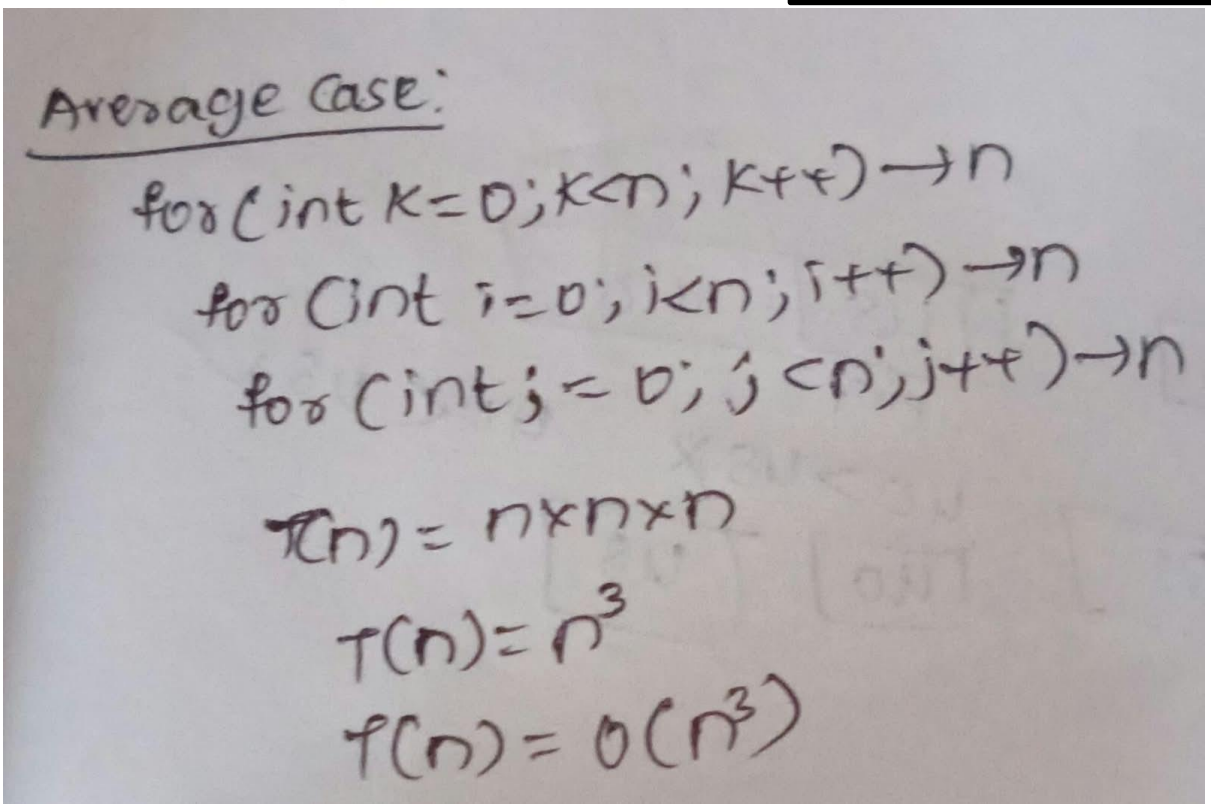
```
Enter number of vertices: 3
Enter adjacency matrix (use 99999 for INF):
0 4 99999
99999 0 2
5 99999 0
Shortest distance matrix:
0 4 6
7 0 2
5 9 0
```

Time Complexity:

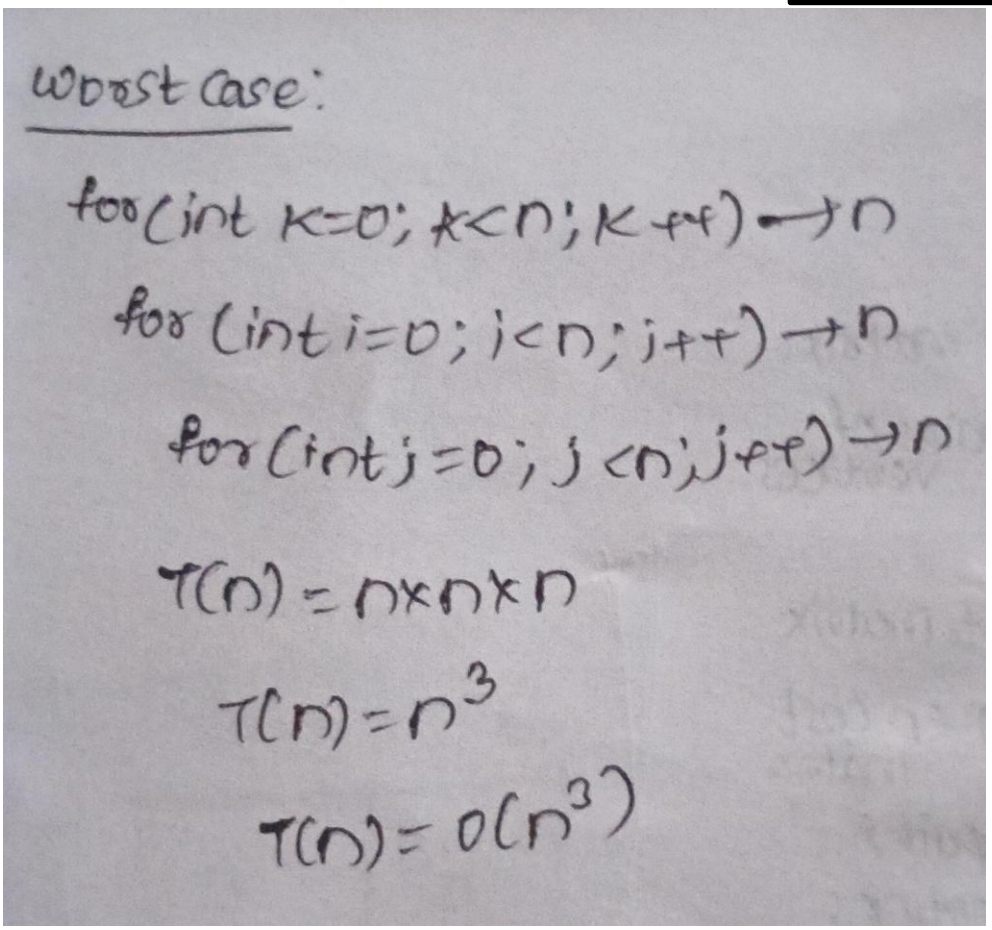
Best Case:



Average Case:


Average Case:
for(int k=0;k<n;k++) → n
for(int i=0;i<n;i++) → n
for(int j=0;j<n;j++) → n
T(n) = n×n×n
T(n) = n³
T(n) = O(n³)

Worst Case:



Conclusion:

The implementation of Floyd–Warshall Algorithm helps us understand how the shortest paths between all pairs of vertices are found in a weighted graph. It updates the distance matrix by considering intermediate vertices and selects the minimum distance between each pair. The algorithm has a time complexity of $O(n^3)$ in the best, average, and worst cases. Therefore, Floyd–Warshall Algorithm is a useful technique for finding the shortest distances between all pairs of vertices in a graph.